COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010/Data Structure and Algorithm	**SECTION:**	CPE21S3
DATE PERFORMED:	August 4, 2026	**DATE SUBMITTED:**	August 9, 2026
NAME:	Gresola, Aryanna Keith G.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 5.1

1. PROCEDURE OUTPUT

ILO A: Create queue using C++ STL

INPUT

ILOA_A_HOA5P1 > main.cpp > display(std::queue<char>)

```cpp
#include <iostream>
#include <queue>

void display(std::queue<char> copyQ);

int main(){
    // create an object:
    std::queue<char> myQ;

    myQ.push('Y');
    myQ.push('A');
    myQ.push('N');
    myQ.push('N');
    myQ.push('A');

    // try all member function:
    // size
    std::cout << "Queue Size: " << myQ.size() << std::endl;
    // front
    std::cout << "Queue Front: " << myQ.front() << std::endl;
    // back
    std::cout << "Queue Back: " << myQ.back() << std::endl;
    // display original:
    std::cout << "Original Queue: ";
    display(myQ);
```

```
    // pop
    myQ.pop();
    std::cout << "After pop operation: ";
    display(myQ);
    // push
    myQ.push('!');
    std::cout << "After push operation: ";
    display(myQ);
    // empty
    if (myQ.empty())
        std::cout << "Queue is empty. " << std::endl;
    else
        std::cout << "Queue is not empty." << std::endl;

    return 0;
}
void display(std::queue<char> copyQ){
    // create a copy of the queue that was passed
    std::queue<char> c = copyQ;
    while(!c.empty()){
        std::cout << " " << c.front();
        c.pop();
    }
    std::cout << std::endl;
}
```

OUTPUT

```
Queue Size: 5
Queue Front: Y
Queue Back: A
Original Queue:  Y A N N A
After pop operation:  A N N A
After push operation:  A N N A !
Queue is not empty.
```

OBSERVATION
From this program, I observed that using the C++ queue library makes queue operations much easier because most of the needed functions are already built in. Instead of creating the queue from scratch, I was able to use functions like push(), pop(), front(), back(), size(), and empty() to manage the data in the queue. It also helped me understand how a queue follows the First-In, First-Out (FIFO) principle, where the first element added is also the first one to be removed. Overall, the queue library makes the code simpler, more organized, and easier to use.

ILO B.1. Linked List Implementation

INPUT (QUEUELL.H)

ILO_B_HOA5P1 > C queueLL.h > ...

```
#ifndef QUEUELL_H
#define QUEUELL_H
#include <iostream>

// create a class
template <typename T>
class qNode{
    public:
        T data;
        qNode<T>* next;
};

// creates the new node
template <typename T>
qNode<T>* new_node(T newData){
    qNode<T>* newNode = new qNode<T>;
    newNode->data = newData;
    newNode->next = nullptr;
    return newNode;
}

// enqueue operation
template <typename T>
void enqueue(T newData, qNode<T>** frontPtr, qNode<T>** backPtr){
    // creates the new node
    qNode<T>* newNode = new_node(newData);

    // inserting into an empty queue:
    if((*frontPtr) == nullptr && (*backPtr) == nullptr){
        (*frontPtr) = newNode;
        (*backPtr) = newNode;
        return;
    }

    // inserting an item into a non-empty queue
    (*backPtr)->next = newNode;
    (*backPtr) = newNode;
}
```

```cpp
// dequeue operation
template <typename T>
void dequeue(qNode<T>** frontPtr, qNode<T>** backPtr){
    //check first if empty:
    if((*frontPtr) == nullptr){
        std::cout << "Queue is empty." << std::endl;
        return;
    }
    // create a temporary node
    qNode<T>* deleteNode = nullptr; // temp variable
    deleteNode = *frontPtr;

    // check if the queue has only 1 node
    if((*frontPtr)->next == nullptr && (*backPtr)->next == nullptr){
        (*frontPtr) = nullptr;
        (*backPtr) = nullptr;
        delete deleteNode;
        return;
    }

    // delete the node
    (*frontPtr) = (*frontPtr)->next;
    deleteNode->next = nullptr;
    delete deleteNode;
}

// check the front
template <typename T>
void checkFront(qNode<T>** frontPtr){
    if((*frontPtr) == nullptr){ // checking if there's an existing front queue
        std::cout << "Front queue is empty." << std::endl;
        return;
    }
    std::cout << "Front: " << (*frontPtr)->data << std::endl;
}

// check the back
template <typename T>
void checkBack(qNode<T>** backPtr){
    if((*backPtr) == nullptr){
        std::cout << "Back queue is empty. " << std::endl;
        return;
```

```
81          return;
82      }
83      std::cout << "Back: " << (*backPtr)->data << std::endl;
84  }
85
86  // display all
87  template <typename T>
88  void display(qNode<T>** frontPtr){
89      qNode<T>* temp = *frontPtr;
90      std::cout << "Queue: ";
91
92      if(temp == nullptr){
93          std::cout << "Queue is empty. " << std::endl;
94          return;
95      }
96      while(temp != nullptr){
97          std::cout << temp->data << " ";
98          temp = temp->next;
99      }
100     std::cout << std::endl;
101
102 }
103
104 #endif
```

INPUT (MAIN.CPP)

ILO_B_HOA5P1 > main.cpp > ...

```cpp
#include <iostream>
#include "queueLL.h"

int main(){
    // use front and back becasue this is a queue implementation
    qNode<char> *front = nullptr;
    qNode<char> *back = nullptr;

    // enqueue
    enqueue('A', &front, &back);
    std::cout << front->data << " " << back->data <<std::endl;
    enqueue('R', &front, &back);
    std::cout << front->data << " " << back->data <<std::endl;
    enqueue('Y', &front, &back);
    std::cout << front->data << " " << back->data <<std::endl;
    enqueue('A', &front, &back);
    std::cout << front->data << " " << back->data <<std::endl;

    // dequeue
    dequeue(&front, &back);
    std::cout << front->data << " " << back->data <<std::endl;


    // checking front
    checkFront(&front);
    checkBack(&back);
    display(&front);

    return 0;
}
```

OUTPUT

```
A A
A R
A Y
A A
R A
Front: R
Back: A
Queue: R Y A
```

OBSERVATION
From this activity, I learned how templates make the queue implementation more flexible since the same functions can be used for different data types without rewriting the code. I was able to create template functions for queue operations such as enqueue(), dequeue(), checkFront(), checkBack(), and display(), which made it easier to access, update, and display the contents of the queue while keeping the code organized and efficient. I also encountered errors such as missing symbols, incorrect syntax, and typographical mistakes, which affected the overall functionality of the program.

ILO B.2. Array-Based Implementation

INPUT (QUEUEARR.H)

ILO_B_HOA5P1 > C queueArr.h > Queue<T>

```
#ifndef QUEUEARRAY_H
#define QUEUEARRAY_H
#include <iostream>

template <typename T>
class Queue{
private:
    // data members:
    T* q_array;
    int q_capacity;
    int q_size;
    int q_front;
    int q_back;

public:
    // constructor
    Queue(int capacity){
        q_capacity = capacity;
        q_size = 0;
        q_front = 0;
        q_back = -1;
        q_array = new T[q_capacity];
    }

    // empty operation:
    bool empty() const{
        if (q_size == 0){
            return true;
        }
        return false;
    }

    // size operation:
    int size() const{
        return q_size;
    }

    // clear operation:
    void clear(){
        q_size = 0;
        q_front =  0;
        q_back = -1;
    }
```

```
44
45      // front operation:
46      T front() const{
47          if (empty()){
48              std::cout << "Queue is empty." << std::endl;
49              return T();
50          }
51          return q_array[q_front];
52      }
53
54      // back operation:
55      T back() const{
56          if (empty()){
57              std::cout << "Back queue is empty." << std::endl;
58              return T();
59          }
60          return q_array[q_back];
61      }
62
63      // enqueue operation:
64      void enqueue(T newData){
65          // checks if the queue is full
66          if (q_size == q_capacity){
67              std::cout << "Queue is full." << std::endl;
68              return;
69          }
70          // inserting into an empty queue
71          if (q_size == 0){
72              q_front = 0;
73              q_back = 0;
74          }else{
75              // circular
76              q_back = (q_back + 1) % q_capacity;
77          }
78          q_array[q_back] = newData;
79          q_size++;
80      }
```

```
81
82      // dequeue operation:
83      void dequeue(){
84          // check if queueu is empty
85          if (empty()){
86              std::cout << "Queue is empty." << std::endl;
87              return;
88          }
89          // queue (1 item)
90          if (q_size == 1){
91              q_front = 0;
92              q_back = -1;
93              q_size = 0;
94              return;
95          }
96          // move in front
97          q_front = (q_front + 1) % q_capacity;
98          q_size--;
99      }
100
101     // copy constructor operator:
102     Queue(const Queue& other){
103         q_capacity = other.q_capacity;
104         q_size = other.q_size;
105         q_front = other.q_front;
106         q_back = other.q_back;
107
108         q_array = new T[q_capacity];
109         // copy the entire array
110         for (int i = 0; i < q_capacity; i++){
111             q_array[i] = other.q_array[i];
112         }
113     }
114
115     // copy assignment operator:
116     Queue& operator=(const Queue& other){
117         // check if it's assigning to itself
118         if (this == &other){
119             return *this;
120         }
```

```
121         // delete existing array
122         delete[] q_array;
123         q_capacity = other.q_capacity;
124         q_size = other.q_size;
125         q_front = other.q_front;
126         q_back = other.q_back;
127         // create new array
128         q_array = new T[q_capacity];
129         // copy the entire array
130         for (int i = 0; i < q_capacity; i++){
131             q_array[i] = other.q_array[i];
132         }
133         return *this;
134     }
135
136     // destructor operator
137     ~Queue(){
138         delete[] q_array;
139     }
140 };
141
142 #endif
143
```

INPUT (MAIN.CPP)

ILO_B_HOA5P1 > main2.cpp > main()

```
1  #include <iostream>
2  #include "queueArr.h"
3
4  int main(){
5
6      int capacity;
7      std::cout << "Enter queue capacity: ";
8      std::cin >> capacity;
9      // constructor
10     Queue<char> queue(capacity);
11     std::cout << "CONSTRUCTOR" << std::endl;
12     std::cout << "Queue created successfully." << std::endl;
13
14     // empty:
15     std::cout << "Is Queue empty? " << (queue.empty()? "Yes" : "No") << std::endl;
16
17     // size
18     std::cout << "Size: " << queue.size() << std::endl;
19     std::cout << std::endl;
```

```
    // enqueue
    std::cout << "ENQUEUE" << std::endl;
    queue.enqueue('A');
    std::cout << queue.front() << " " << queue.back() << std::endl;
    queue.enqueue('R');
    std::cout << queue.front() << " " << queue.back() << std::endl;
    queue.enqueue('Y');
    std::cout << queue.front() << " " << queue.back() << std::endl;
    queue.enqueue('A');
    std::cout << queue.front() << " " << queue.back() << std::endl;

    // checking front:
    std::cout << "Front: " << queue.front() << std::endl;

    // checking back:
    std::cout << "Back: " << queue.back() << std::endl;

    // checking size:
    std::cout << "Size: " << queue.size() << std::endl;
    std::cout << std::endl;


    // dequeue
    std::cout << "DEQUEUE" << std::endl;
    queue.dequeue();
    std::cout << "Front after dequeue: " << queue.front() << std::endl;
    std::cout << "Back after dequeue: " << queue.back() << std::endl;
    std::cout << "Size after dequeue: " << queue.size() << std::endl;
    std::cout << std::endl;

    // testing circular queue:
    std::cout << "CIRCULAR QUEUE TEST" << std::endl;
    queue.enqueue('N');
    std::cout << "Front: " << queue.front() << std::endl;
    std::cout << "Back: " << queue.back() << std::endl;
    std::cout << "Size: " << queue.size() << std::endl;
    std::cout << std::endl;

    // copy constructor:
    std::cout << "COPY CONSTRUCTOR" << std::endl;
    Queue<char> copiedQueue(queue);
    std::cout << "Copied Queue Front: " << copiedQueue.front() << std::endl;
    std::cout << "Copied Queue Back: " << copiedQueue.back() << std::endl;
    std::cout << "Copied Queue Size: " << copiedQueue.size() << std::endl;
    std::cout << std::endl;
```

```cpp

    // copy assignment operator:
    std::cout << "COPY ASSIGNMENT OPERATOR" << std::endl;
    Queue<char> assignedQueue(capacity);
    assignedQueue = queue;

    std::cout << "Assigned Queue Front: " << assignedQueue.front() << std::endl;
    std::cout << "Assigned Queue Back: " << assignedQueue.back() << std::endl;
    std::cout << "Assigned Queue Size: " << assignedQueue.size() << std::endl;
    std::cout << std::endl;


    // clear:
    std::cout << "CLEAR" << std::endl;
    queue.clear();
    std::cout << "Size After Clear: " << queue.size() << std::endl;
    std::cout << "Is Queue empty? " << (queue.empty()? "Yes" : "No") << std::endl;
    std::cout << std::endl;

    // destructor
    std::cout << "DESTRUCTOR" << std::endl;
    std::cout << "It will automatically release." << std::endl;

    return 0;
}
```

OUTPUT

```
Enter queue capacity: 5
CONSTRUCTOR
Queue created successfully.
Is Queue empty? Yes
Size: 0

ENQUEUE
A A
A R
A Y
A A
Front: A
Back: A
Size: 4

DEQUEUE
Front after dequeue: R
Back after dequeue: A
Size after dequeue: 3

CIRCULAR QUEUE TEST
Front: R
Back: N
Size: 4
```

```
COPY CONSTRUCTOR
Copied Queue Front: R
Copied Queue Back: N
Copied Queue Size: 4

COPY ASSIGNMENT OPERATOR
Assigned Queue Front: R
Assigned Queue Back: N
Assigned Queue Size: 4

CLEAR
Size After Clear: 0
Is Queue empty? Yes

DESTRUCTOR
It will automatically release.
```

OBSERVATION

For this activity, the queue was implemented using an array instead of a linked list. The elements are stored in a dynamically allocated array and managed using q_front, q_back, and q_size, with a circular approach to reuse spaces. I still used separate header and main files because it keeps the code more organized for me, where I define the operations in the header and call and test them in the main. I also noticed that I needed to be more careful with the indexes and capacity because the front and back positions need to be updated correctly during enqueue, dequeue, and circular operations to avoid accessing the wrong position or exceeding the array capacity. The copy constructor and copy assignment also helped me create and assign queues with the same contents while keeping their arrays separate.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO C: Solve problems using queue implementation

Problem Title: Shared Printer Simulation using Queues

INPUT (PRINTER.H)

ILO_C_HOA5P1 > C printer.h > Printer<T> > displayQueue()

```
1   #ifndef PRINTER_H
2   #define PRINTER_H
3   #include <iostream>
4   #include <string>
5
6   class Job{
7   public:
8       int id;
9       std::string user;
10      int pages;
11
12      Job(){
13          id = 0;
14          user = " ";
15          pages = 0;
16      }
17
18      Job(int i, std::string u, int p){
19          id = i;
20          user = u;
21          pages = p;
22      }
23  };
24
25  template <class T>
26  class Node{
27  public:
28      T data;
29      Node<T>* next;
30
31      Node(T value){
32          data = value;
33          next = nullptr;
34      }
35  };
```

```cpp
template <class T>
class Printer{
private:
    Node<T>* front;
    Node<T>* rear;
public:
    // constructor
    Printer(){
        front = nullptr;
        rear = nullptr;
    }
    // checking if the queue is empty
    bool isEmpty(){
        return front == nullptr;
    }
    // add print job
    void addJob(T job){
        Node<T>* newNode = new Node<T>(job);
        if (isEmpty()){
            front = newNode;
            rear = newNode;
        }else{
            rear->next = newNode;
            rear = newNode;
        }
        std::cout << std::endl;
        std::cout << "Print Job Added Successfully!" << std::endl;
        std::cout << "Job ID: " << job.id << std::endl;
        std::cout << "User: " << job.user << std::endl;
        std::cout << "Pages: " << job.pages << std::endl;
    }
    // display queue
    void displayQueue(){
        if (isEmpty()){
            std::cout << "Printer Queue is empty." << std::endl;
            return;
        }
        Node<T>* temp = front;
        std::cout << "--- PENDING PRINT JOBS ---" << std::endl;

        while (temp != nullptr){
            std::cout << "Job ID: " << temp->data.id << std::endl;
            std::cout << "User: " << temp->data.user << std::endl;
            std::cout << "Pages: " << temp->data.pages << std::endl;
            std::cout << std::endl;
```

```
82
83              temp = temp->next;
84          }
85      }
86      // search job
87      void searchJob(int id){
88          if (isEmpty()){
89              std::cout << "Printer Queue is empty." << std::endl;
90              return;
91          }
92          Node<T>* temp = front;
93
94          while (temp != nullptr){
95              if (temp->data.id == id){
96                  std::cout << "Job Found!" << std::endl;
97                  std::cout << "Job ID: " << temp->data.id << std::endl;
98                  std::cout << "User: " << temp->data.user << std::endl;
99                  std::cout << "Pages: " << temp->data.pages << std::endl;
100                 return;
101             }
102             temp = temp->next;
103         }
104         std::cout << "Job ID not found. Please try again.";
105     }
106     // peek next job in queue
107     void peekNextJob(){
108         if (isEmpty()){
109             std::cout << "Printer Queue is empty." << std::endl;
110             return;
111         }
112         std::cout << "--- NEXT PRINT JOB ---" << std::endl;
113         std::cout << "Job ID: " << front->data.id << std::endl;
114         std::cout << "User: " << front->data.user << std::endl;
115         std::cout << "Pages: " << front->data.pages << std::endl;
116     }
117     // printer status
118     void printerStatus(){
119         int jobs = 0;
120         int pages = 0;
```

```
121
122         Node<T>* temp = front;
123         while (temp != nullptr){
124             jobs++;
125             pages += temp->data.pages;
126             temp = temp->next;
127         }
128         std::cout << "--- PRINTER STATUS ---" << std::endl;
129         if (isEmpty()){
130             std::cout << "Status: Idle (Inactive)" << std::endl;
131             std::cout << "Pending Jobs: 0" << std::endl;
132             std::cout << "Total Pages: 0" << std::endl;
133         }else{
134             std::cout << "Status: Busy" << std::endl;
135             std::cout << "Pending Jobs: " << jobs << std::endl;
136             std::cout << "Total Pages: " << pages << std::endl;
137             std::cout << "Next User: " << front->data.user << std::endl;
138         }
139     }
140     // process next job
141     void processNextJob(){
142         if (isEmpty()){
143             std::cout << "Printer Queue is empty." << std::endl;
144             return;
145         }
146         Node<T>* temp = front;
147 
148         std::cout << "--- PRINTING JOB ---" << std::endl;
149         std::cout << "Job ID: " << temp->data.id << std::endl;
150         std::cout << "User: " << temp->data.user << std::endl;
151         std::cout << "Pages: " << temp->data.pages << std::endl;
152         std::cout << "Status: Completed" << std::endl;
153 
154         front = front->next;
155         delete temp;
156         if (front == nullptr){
157             rear = nullptr;
158         }
159         std::cout << "Job Successfully Printed!" << std::endl;
160     }
```

```cpp
    // process all jobs
    void processJobs(){
        if (isEmpty()){
            std::cout << "Printer Queue is empty." << std::endl;
            return;
        }
        std::cout << "--- PROCESSING ALL PRINT JOBS --- " << std::endl;
        while (front != nullptr){
            std::cout << "Printing Job..." << std::endl;
            std::cout << "Job ID: " << front->data.id << std::endl;
            std::cout << "User: " << front->data.user << std::endl;
            std::cout << "Pages: " << front->data.pages << std::endl;
            std::cout << "Status: Completed" << std::endl;

            Node<T>* temp = front;
            front = front->next;
            delete temp;
        }
        rear = nullptr;
        std::cout << "All Print Jobs Have Been Processed." << std::endl;
    }
};

#endif
```

INPUT (MAIN.CPP)

ILO_C_HOA5P1 > main.cpp > main()

```cpp
#include <iostream>
#include <string>
#include "printer.h"

int main(){
    Printer<Job> printer;

    int choice;
    int nextJobID = 101;

    do{
        std::cout << std::endl;
        std::cout << "--- SHARED PRINTER SYSTEM ---" << std::endl;
        std::cout << "1. Add Print Job" << std::endl;
        std::cout << "2. Display Queue" << std::endl;
        std::cout << "3. Search Job" << std::endl;
        std::cout << "4. Peek Next Job" << std::endl;
        std::cout << "5. Printer Status" << std::endl;
        std::cout << "6. Process Next Job" << std::endl;
        std::cout << "7. Process All Jobs " << std::endl;
        std::cout << "8. Exit" << std::endl;
        std::cout << "---------------------------------" << std::endl;
        std::cout << "Enter your choice: ";
        std::cin >> choice;
```

```
    switch (choice){
        case 1: {
            std::string user;
            int pages;

            std::cout << "--- ADD PRINT JOB --- " << std::endl;
            std::cout << "Assigned Job ID: " << nextJobID << std::endl;

            std::cin.ignore();

            std::cout << "Enter Username: ";
            std::getline(std::cin, user);
            std::cout << "Enter number of pages: ";
            std::cin >> pages;

            Job job(nextJobID, user, pages);
            printer.addJob(job);
            nextJobID++;
            break;
        }
        case 2:
            printer.displayQueue();
            break;
        case 3: {
            int id;
            std::cout << "Enter Job ID to search: ";
            std::cin >> id;
            printer.searchJob(id);
            break;
        }
        case 4:
            printer.peekNextJob();
            break;
        case 5:
            printer.printerStatus();
            break;
        case 6:
            printer.processNextJob();
            break;
        case 7:
            printer.processJobs();
        case 8:
            std::cout << "Exiting Shared Printer System..." << std::endl;
            break;
```

```
            default:
                std::cout << "Invalid choice. Please try again." << std::endl;
        }

    } while (choice != 8);

    return 0;
}
```

OUTPUT

ADD PRINT JOB

```
--- SHARED PRINTER SYSTEM ---
1. Add Print Job
2. Display Queue
3. Search Job
4. Peek Next Job
5. Printer Status
6. Process Next Job
7. Process All Jobs
8. Exit
---------------------------------
Enter your choice: 1
--- ADD PRINT JOB ---
Assigned Job ID: 101
Enter Username: Audrey
Enter number of pages: 5

Print Job Added Successfully!
Job ID: 101
User: Audrey
Pages: 5
```

DISPLAY QUEUE

```
--- SHARED PRINTER SYSTEM ---
1. Add Print Job
2. Display Queue
3. Search Job
4. Peek Next Job
5. Printer Status
6. Process Next Job
7. Process All Jobs
8. Exit
---------------------------------
Enter your choice: 2
--- PENDING PRINT JOBS ---
Job ID: 101
User: Audrey
Pages: 5

Job ID: 102
User: Aryanna
Pages: 10
```

OUTPUT

SEARCH JOB (ID FOUND)

```
--- SHARED PRINTER SYSTEM ---
1. Add Print Job
2. Display Queue
3. Search Job
4. Peek Next Job
5. Printer Status
6. Process Next Job
7. Process All Jobs
8. Exit
----------------------------------
Enter your choice: 3
Enter Job ID to search: 101
Job Found!
Job ID: 101
User: Audrey
Pages: 5
```

SEARCH JOB (ID NOT FOUND)

```
--- SHARED PRINTER SYSTEM ---
1. Add Print Job
2. Display Queue
3. Search Job
4. Peek Next Job
5. Printer Status
6. Process Next Job
7. Process All Jobs
8. Exit
----------------------------------
Enter your choice: 3
Enter Job ID to search: 100
Job ID not found. Please try again.
```

OUTPUT

PEEK NEXT JOB

```
--- SHARED PRINTER SYSTEM ---
1. Add Print Job
2. Display Queue
3. Search Job
4. Peek Next Job
5. Printer Status
6. Process Next Job
7. Process All Jobs
8. Exit
----------------------------------
Enter your choice: 4
--- NEXT PRINT JOB ---
Job ID: 101
User: Audrey
Pages: 5
```

PRINTER STATUS (BUSY)

```
--- SHARED PRINTER SYSTEM ---
1. Add Print Job
2. Display Queue
3. Search Job
4. Peek Next Job
5. Printer Status
6. Process Next Job
7. Process All Jobs
8. Exit
----------------------------------
Enter your choice: 5
--- PRINTER STATUS ---
Status: Busy
Pending Jobs: 2
Total Pages: 15
Next User: Audrey
```

OUTPUT

PRINTER STATUS (IDLE)

```
--- SHARED PRINTER SYSTEM ---
1. Add Print Job
2. Display Queue
3. Search Job
4. Peek Next Job
5. Printer Status
6. Process Next Job
7. Process All Jobs
8. Exit
----------------------------------
Enter your choice: 5
--- PRINTER STATUS ---
Status: Idle (Inactive)
Pending Jobs: 0
Total Pages: 0
```

PROCESS NEXT JOB

```
--- SHARED PRINTER SYSTEM ---
1. Add Print Job
2. Display Queue
3. Search Job
4. Peek Next Job
5. Printer Status
6. Process Next Job
7. Process All Jobs
8. Exit
----------------------------------
Enter your choice: 6
--- PRINTING JOB ---
Job ID: 101
User: Audrey
Pages: 5
Status: Completed
Job Successfully Printed!
```

OUTPUT

PROCESS ALL JOBS

```
--- SHARED PRINTER SYSTEM ---
1. Add Print Job
2. Display Queue
3. Search Job
4. Peek Next Job
5. Printer Status
6. Process Next Job
7. Process All Jobs
8. Exit
----------------------------------
Enter your choice: 7
--- PROCESSING ALL PRINT JOBS ---
Printing Job...
Job ID: 102
User: Aryanna
Pages: 10
Status: Completed
All Print Jobs Have Been Processed.
Exiting Shared Printer System...
```

EXIT

```
--- SHARED PRINTER SYSTEM ---
1. Add Print Job
2. Display Queue
3. Search Job
4. Peek Next Job
5. Printer Status
6. Process Next Job
7. Process All Jobs
8. Exit
----------------------------------
Enter your choice: 8
Exiting Shared Printer System...
```

OBSERVATION

I observed that in this program, using a linked list for the queue made managing multiple print jobs more flexible because jobs can be added and removed without having a fixed capacity like an array. I also found that separating the program into a header file and a main file made my code more organized, since I could place the classes and functions in the header and call them when needed in the main file. I added eight features in total: add print job, display queue, search job ID, peek next job, printer status, process next job, process all jobs, and exit. I think these features are useful in a shared printer system because they make it easier to track the jobs in the queue and avoid confusion when multiple print requests are being handled. I really enjoyed doing this activity because I was able to customize the system based on what I think would be useful, and it also helped me get the hang of linked lists and understand better how they can be used to organize and manage data in a queue.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: ChatGPT

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: Explanation of concepts

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: I used ChatGPT mainly for concept explanations to better understand the topics. However, it did not replace my authorship, analysis, original thinking, or hands-on work in the activity.

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.